Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Algoritmi de aproximare (1) -

Algoritmi de aproximare

– se folosesc pentru "rezolvarea aproximativă" a problemelor dificile de optimizare (corespunzătoare unor probleme de decizie NP-dure sau NP-complete).

Def: Un algoritm de aproximare pentru o problemă Q este un algoritm determinist tractabil care duce la soluții neoptimale ale problemei, dar cu un grad de calitate garantat în raport cu optimul.

Def: Fie Q o problemă cu date într-o mulțime I, iar $cost^*(i)$ este costul soluției optime și $cost(i)$ este costul soluției găsite de un algoritm de aproximare pentru problema Q, pentru datele $i \in I$. Notăm cu $\bar{i}$, $\bar{i} \in \mathbb{N}^*$, dimensiunea datelor $i \in I$.

Factorul de aproximare al algoritmului de aproximare este:

$$\delta(n) = \max_{\substack{i \in I \\ \bar{i} = n}} \frac{cost(i)}{cost^*(i)}$$, dacă problema este de minimizare

$$\delta(n) = \max_{\substack{i \in I \\ \bar{i} = n}} \frac{cost^*(i)}{cost(i)}$$, dacă problema este de maximizare

Obs: $\delta(n) \geq 1$, $\forall n$

Obs: Dacă $f(n)$ este mărginit superior de o constantă pentru $\forall n$ atunci factorul de aproximare se notează cu δ și are forma:

$$\delta = \max_{i \in I} \frac{cost(i)}{cost^*(i)}$$, dacă problema este de minimizare

$$\delta = \max_{i \in I} \frac{cost^*(i)}{cost(i)}$$, dacă problema este de maximizare

Obs: $\delta \geq 1$.

Problema λ - arc conectivitate

Def: Fie $G = (V, E)$ un graf neorientat și conex cu n noduri, iar λ un număr întreg, $2 \leq \lambda \leq n-1$. G s.n. **λ - arc conectat** dacă prin eliminarea oricăror $\lambda - 1$ arce din G se obține un graf conex.

Def: S.n. **λ-arc conectivitate** problema:

" Fie $G = (V, E)$ un graf neorientat, și λ-arc conectat cu n noduri, unde λ este un număr întreg, $2 \leq \lambda \leq n-1$. Să se determine un subgraf de acoperire $G_\lambda = (V, E_\lambda)$ al lui G, cu număr minim de arce, astfel încât G_λ să fie λ-arc conectat!"

Algoritmul de aproximare Khuller-Vishkin

(algoritmul rezolvă problema λ-arc conectivitate)

a) Construcția unui subgraf de acoperire 2-arc conectat

$G = (V, E)$: graful inițial, 2-arc conectat;

$G_2 = (V, E_2)$, $E_2 \subseteq E$: graful final, 2-arc conectat;

$Arb(r)$: arborele cu rădăcina r construit în cursul parcurgerii în adâncime a lui G;

$d(u)$: momentul descoperirii nodului u în cursul parcurgerii în adâncime a grafului G;

$succ_G(u)$: nodurile adiacente nodului u în graful G;

$\Pi(u)$: predecesorul direct al nodului u în $Arb(r)$;

$c(u)$: culoarea nodului u;

$b(u) = \min\{ d(v) \mid v$ este redescoperit pornind din u în cursul explorării grafului $G\}$;

$b_2(u) = \min\{ d(v) \mid v$ este redescoperit pornind din u în cursul explorării grafului $G_2\}$;

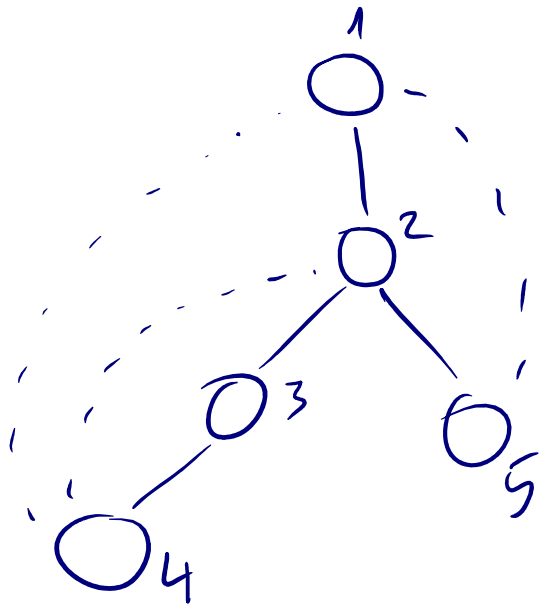
$e(u)$: arc invers (x, y) astfel încât y este predecesor al lui x în $Arb(r)$, iar u este pe calea $x, \pi(x), \pi(\pi(x)), \ldots, y$ în $Arb(r)$.

Arce de arbore și arce inverse

Arc de arbore (arc din arbore):
arc care face parte din arborele obținut în urma parcurgerii în adâncime a grafului grafului;

Arc invers: arc prin care se redescoperă un nod în cursul parcurgerii în adâncime a grafului.

Ex:



1, 2, 3, 4, 5:
ordinea parcurgerii nodurilor în metoda parcurgerii în adâncime

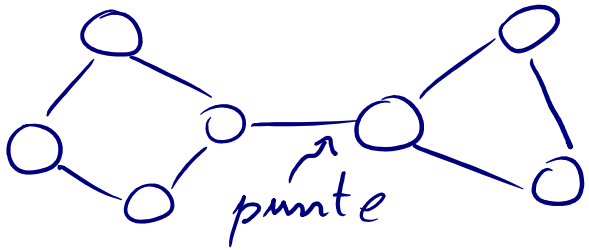
— arce de arbore

- - - arce inverse

```
arc_2_conectat(G)
    for-each u ∈ V
        c(u) = alb
        π(u) = null
    timp = 0
    E_2 = ∅
    fie r ∈ V ales la întâmplare
    explorare(r)
    return E_2
```

Punte

= arc prin eliminarea căruia graful se deconectează.



explorare (u)

d(u) = b(u) = b_2(u) = + + timp

l(u) = (u, u) // arc invers de aceeași a pentru arborele din G_2

c(u) = cenușiu

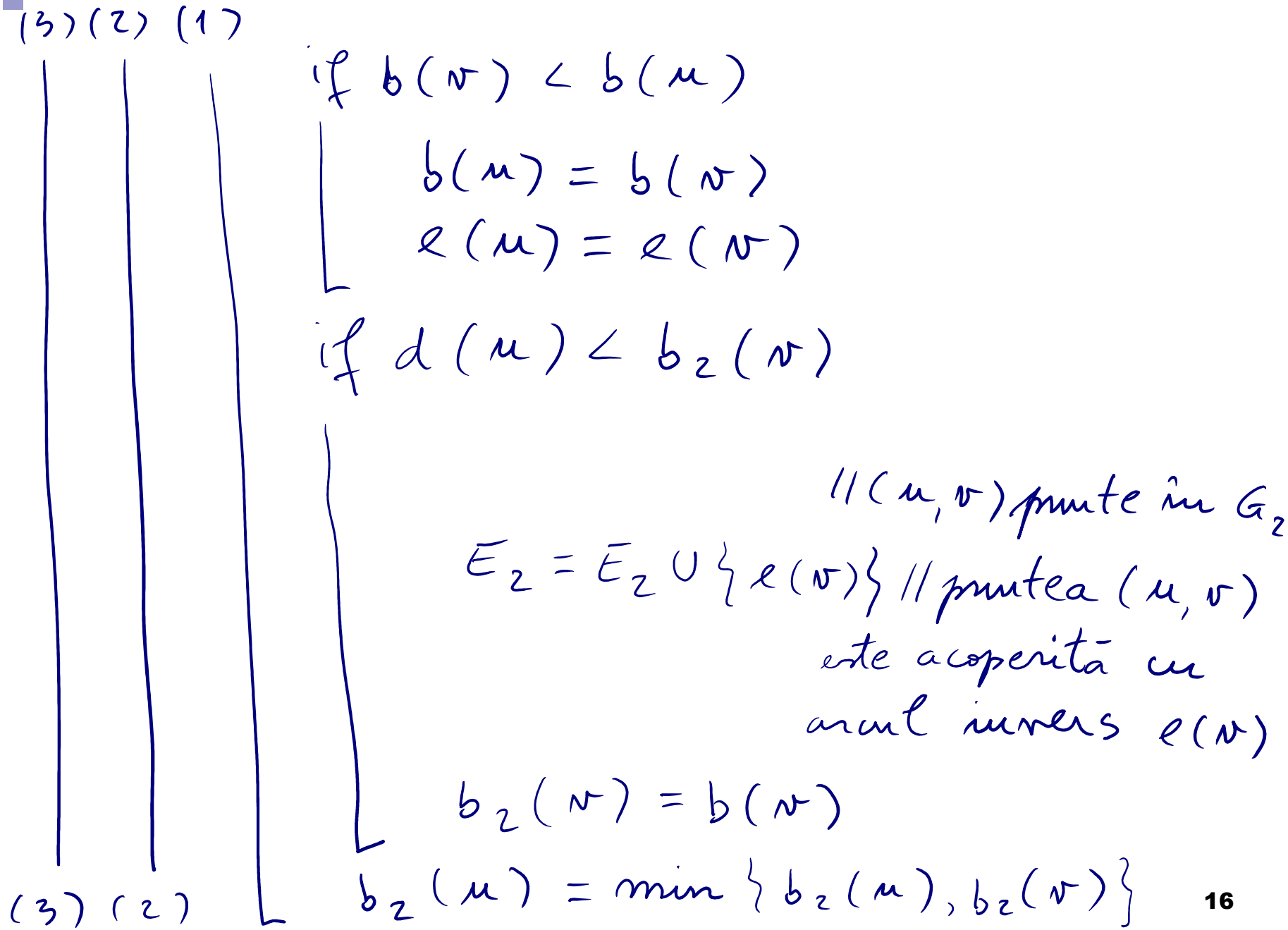
for - each v ∈ succs(u)

(1) if c(v) = alb

(2) π(v) = u

(3) E_2 = E_2 ∪ {(u, v)} // (u, v) arc de arbore

explorare (v)


(3) (2) (1)
if b(v) < b(u)
b(u) = b(v)
e(u) = e(v)
if d(u) < b2(v)
// (u, v) punte în G2
E2 = E2 ∪ {e(v)} // punteaza (u, v)
este acoperită cu
arcul invers e(v)
b2(v) = b(v)
b2(u) = min {b2(u), b2(v)}
(3) (2)

(3) (2) else // v este redescoperit

if $\pi(u) \neq v$ // (u, v) arc invers

if $d(v) < b(u)$

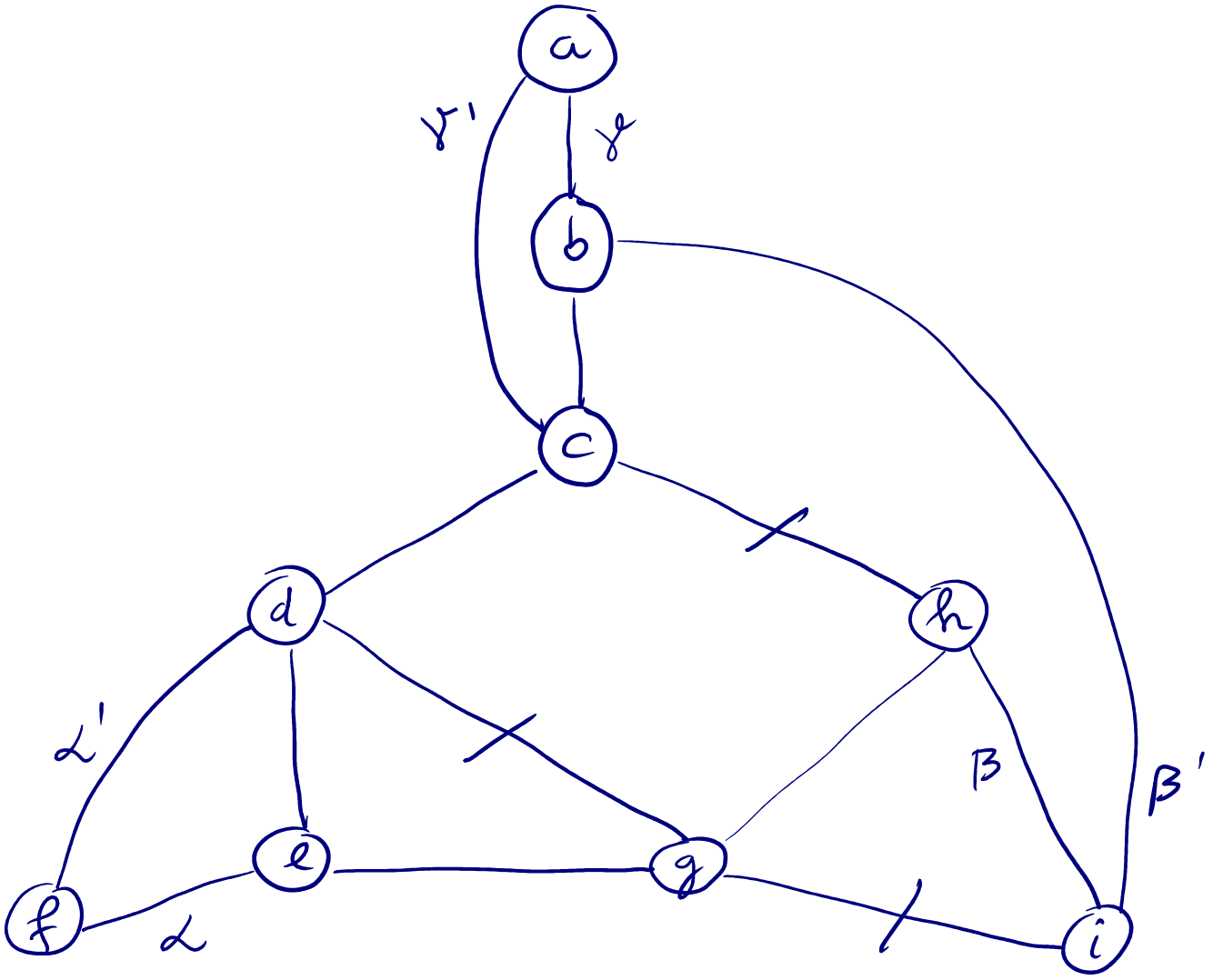
$b(u) = d(v)$

$e(u) = (u, v)$

$c(u) =$ negru

Complexitatea temporală a algoritmului arc_2_conectat este aceeași cu a algoritmului de parcurgere în adâncime: $O(n+m)$, unde $n = card(V)$ și $m = card(E)$.

Ex :



Explicații pt Ex:

- în graful inițial G sunt toate nodurile și toate arcele din desen;
- în graful final G_2 sunt toate nodurile din desen și toate arcele netăiate din desen;
- modul de start al parcurgerii în adâncime a grafului este a;
- ordinea vizitării nodurilor: a, b, c, d, e, f, g, h, i;
- punți: $\alpha = (e, f)$, $\beta = (h, i)$, $\gamma = (a, b)$;
- arce inverse: $\alpha' = (f, d)$, $\beta' = (i, b)$, $\gamma' = (c, a)$.

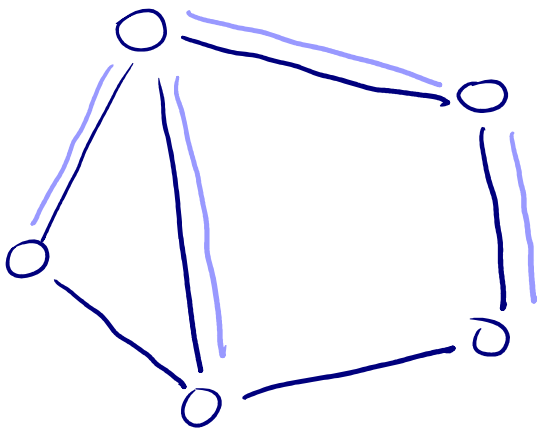
b) Construcția unui subgraf de acoperire λ - arc conectat

- se pleacă de la un subgraf de acoperire $G_2 = (V, E_2)$ 2-arc conectat al lui G;
- se adaugă iterativ noi arce din E la arcele din E_2, la fiecare pas mărindu-se cu 1 factorul de conectivitate al subgrafului;
- în final, G_λ va fi subgraf de acoperire λ - arc conectat.

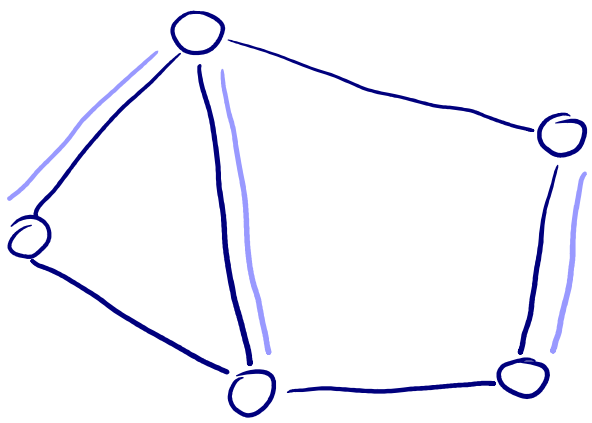
Pădure de acoperire a unui graf =

= mulțime de arbori care acoperă nodurile acelui graf, fiecare nod fiind acoperit o singură dată.

Ex:



arbore de acoperire



pădure de acoperire

```
arc_λ_conectat(G, λ)
    E_2 = arc_2_conectat(G)
    // construieste subgrafuri G_{i+1} = (V, E_{i+1})
    // (i+1) - arc conectate, i = 2, λ-1
    for i = 2 .. λ-1
        E' = E \ E_i
        A_{i+1} = arce_acoperire(V, E')
        // A_{i+1} sunt arcele padurii de acoperire
        // a grafului (V, E')
        E_{i+1} = E_i ∪ A_{i+1}
    return E_λ
```

```
arce_acoperire (V, E')
    // parcurgere în adâncime a grafului (V, E')
    for-each u ∈ V
        c(u) = alb
    A = ∅
    for-each u ∈ V
        if c(u) = alb
            culege_arce (u, E')
    return A
```

```
culege_arce(u, E')
    c(u) = cenusie
    for-each v ∈ succs(u, E')
        if c(v) = alb
            A = A ∪ {(u, v)}
            culege_arce(v, E')
```

Complexitatea temporală a algoritmului arc_λ_conectat este determinată de complexitatea algoritmului arc_2_conectat și de cele λ − 2 parcurgeri în adâncime ale grafului:

$O((\lambda-1)\cdot(n+m))$, $n = card(V)$, $m = card(E)$.

(T) Factorul de aproximare pentru algoritmul arc_λ_conectat este $\delta < 2 - \frac{1}{\lambda}$.